

Applied AI Engineer Assignment – AI-Powered Market Intelligence

Name : Vishnu Vardhan P R

Email : rajagopalvishnu8@gmail.com

Phone Number : 8939149650

Linkedin : <https://www.linkedin.com/in/vishnu-varadhan-p-r-75b643267/>

Dear Hiring Manager,

This is the manual documentation for this whole assignment. I'm writing all the details with all the steps followed in this documentation. And I'm also writing the steps to follow to setup the app locally and run it.

CLEANING AND NORMALIZING THE 'Google Play Store Apps' FROM KAGGLE:

1. The first step of this phase of the project involved downloading the dataset from kaggle with the link provided in the pdf.
2. After downloading, I opened the dataset named **googleplaystore.csv** and imported necessary libraries and started exploring the dataset, both visually and with values.
3. I first noticed the values not being in a standard format. Like numerical values had an object data type. So I focused on changing the data types first. And some of the fields even had extra unnecessary characters in them (like : "+" or " "). So careful observation and fixing measures were taken.
4. After fixing the data types of the columns, I noticed the null values present.
5. Some fields had less null values and others had higher percentages, so I first checked if there is any pattern in that missing null values. For this I used seaborn for heatmaps and correlation matrices. But then I did not notice a pattern so it was okay.
6. After that I decided to fill the null values either by taking the median or dropping the column or putting a zero value. All these depended on the type of the field and the number of null values.
7. Then I had to standardize a few fields like the size field, which had sizes in different units. So I made them into MB and stored them as float values.
8. Even fields like Genre and Category had multiple entries for a single row's field, so I had to standardize them also.
9. Doing these steps, the dataset was pretty much clean with no null values and normalized columns. I had a few unnecessary columns, which I then dropped.
10. The **Cleaning playstore dataset.ipynb** has all the steps I mentioned and taking a look at that would prove my work.
11. The cleaned dataset was stored as **cleaned_googleplaystore.csv** .

INTEGRATING 'App Store Scraper API', HANDLING RATE LIMITS AND RETRIES AND COMBINING THE APPSTORE DATA WITH THE PLAYSTORE DATA:

1. First I went to the site mentioned "[Rapidapi.com](https://rapidapi.com)". Then I Logged in to use the "App Store Scraper API". The api key and host keys were already created for me and I used them in my code to fetch the data.
2. I created a .env file and entered the api details there. Then I created a python file named it **latest_appstore_integ.py** . I wrote a GET wrapper for Rapidapi with retries to fetch the data. I used the requests library of python to help me with this.
3. The get wrapper function had a parameters argument, which would change according to the need of the api.
4. If the request is for searching the app, then params would require the app query, language, numbers etc. And if they request it to fetch the app details, it would require just the app id. My function would do both for me
5. In the main loop, I collected the playstore dataset's app names and ran the get wrapper with the app name to find the app, then I used the other api to fetch the app details. The loop did this for all the app names from the playstore apps dataset.
6. Then the loop also used a function to transform the app's details into a dictionary and then appended it to the variable holding the results of every app's details.
7. Since there were rate limits, I had to re-run this code many times, and then everytime it hits the limit, the data fetched will get stored to a csv file.
8. I dumped all this data into a csv file named **appstore_apps_by_name_(i).csv** . Where i refers to the i'th run of the code.
9. Then I opened an ipynb file named **Updated Combine Datasets.ipynb** . Where I loaded all the appstore csv files I created in the previous step. I merged all of it by using the pd.concat function to form a single appstore dataset.
10. Then in the ipynb file I cleaned both the playstore data and appstore data to further merge them.
11. I have done a lot of processing, which I have clearly mentioned in the **Updated Combine Datasets.ipynb** file. I made changes to both the dataframes and merged both to the playstore dataframe.
12. After making careful changes in both the dataframes, I merged both the dataframes and then saved them as a new csv file named **updated_cleaned_combined.csv** .
13. This is the csv file which was used in the next phases of the assignment.

IMPLEMENTING CONFIDENCE SCORING AND STATISTICAL VALIDATION FOR INSIGHTS:

1. The first step was to extract basic descriptive insights from the merged Play Store and App Store data. These included metrics such as the best-performing categories, average ratings, and app counts. To structure these findings in a reusable format, the insights were stored as JSON objects. Each JSON entry included the insight's title,

description, supporting statistics, and a recommendation. This ensured consistency and made it easier to extend each insight later with validation results.

2. Rather than relying only on descriptive statistics, each insight was reframed as a testable hypothesis. For example, “Paid apps have higher ratings than free apps” or “Recently updated apps perform better than older apps.” Defining hypotheses in this way allowed the analysis to move beyond observations into statistically testable claims.
3. Once hypotheses were established, the appropriate statistical tests were chosen. Since most insights involved comparing the mean ratings of two independent groups (e.g., free vs paid), the independent samples t-test was selected.
4. Validation functions were then created to perform the tests automatically. Each function carried out the following steps: split the dataset into two groups, run the t-test, and return results in a structured format. The output included the t-statistic, p-value, group averages, and sample sizes. Organizing the output in JSON format ensured consistency with the initial insights file.
5. To make results more intuitive for business users, a confidence score was defined (confidence score = $1 - \text{p-value}$). This transformed statistical significance into a percentage-based score that indicated how strongly the result could be trusted. For instance, a p-value of 0.03 translated into a confidence score of 0.97, meaning there was 97% confidence that the observed difference was real.
6. Several hypotheses were validated using this framework. Tests were run on comparisons such as free vs paid apps, recent vs old apps, small vs large apps, and category-to-category comparisons (e.g., Games vs Productivity). Each validation returned a JSON object with detailed statistics, confidence scores, and recommendations based on whether the p-value indicated statistical significance.
7. Finally, all validated insights were compiled into a single JSON file. This file provided a complete record of each hypothesis, the statistical results, and confidence levels. By saving the output in this structured format, insights could be easily revisited, compared, or included in automated reporting.

INTEGRATING OPENAI TO GENERATE INSIGHTS:

1. The first step was to create an API key from the OpenAI platform. Once generated, the key was stored securely as an environment variable rather than being hardcoded directly into the code. This practice ensured that the key remained safe and the code could be shared or deployed without exposing sensitive credentials.
2. With the key set up, the next step was to configure the OpenAI Python client. This was done by importing the library and linking it to the environment variable containing the API key. Once configured, all subsequent calls to the OpenAI API could be authenticated automatically, making the integration smooth and reusable.
3. To generate clear and professional summaries of insights, an executive summary function was created. This function took the dataframe of the **cleaned_combined.csv** dataset and the structured insights json file, converted it into a textual context, and passed it to the OpenAI model with instructions to produce a summary. The output was a polished, human-readable explanation of the dataset’s key findings, which could be reused in reports or documentation.

4. In addition to summaries, a question–answer function was developed. This function combined the dataframe of the **cleaned_combined.csv** dataset and the json file insights with a user-provided question and used OpenAI to return a concise, business-friendly answer. This allows the users to query the insights dynamically, moving beyond static reporting and enabling interactive exploration of the analysis.
5. The final step was to connect the executive summary function to the automated PDF report generator. After the insights and plots were prepared, the AI-generated summary was inserted into a dedicated “Executive Summary” section at the beginning of the report. This ensured that every report included a high-quality narrative overview of the findings, making the documents more useful.

BUILDING STREAMLIT INTERFACE FOR AS THE DASHBOARD AND THE AUTOMATION OF THE REPORT GENERATION:

1. Once the insights and validation framework were in place, the next step was to create an interactive dashboard and automate the process of generating reports. Streamlit was chosen as the interface because it allows for rapid development of user-friendly applications directly in Python.
2. The first step involved installing and configuring Streamlit. With the package added to the project dependencies, a dedicated Python script was created to serve as the application entry point. Streamlit provided an easy way to connect the cleaned dataset, insights, and analysis functions with an interactive user interface.
3. The core insights from the dataset, along with their statistical validation results, were displayed within the dashboard. Tables, charts, and narrative explanations were included so that users could both see the underlying numbers and understand their meaning. This turned the insights into an interactive experience rather than static text.
4. Visualizations such as top categories, rating distributions, and content rating breakdowns were integrated into the dashboard. These plots were automatically generated using matplotlib and displayed directly within the Streamlit interface. This gave stakeholders a clear picture of the dataset’s structure and patterns in addition to the derived insights.
5. A dedicated section of the dashboard was designed for report automation. After generating insights, validations, and plots, the system compiled everything into a professional PDF report. The process used the ReportLab library to structure the content into sections: cover page, AI-generated executive summary, validated insights, statistical results, and EDA plots. A download option was provided so users could export the report instantly.
6. The AI-generated executive summary, created with OpenAI, was also integrated into the automated report. This ensured that each report not only contained charts and statistics but also a polished narrative overview. Including both automated analysis and AI-written text made the report complete and business-ready.

A few snapshots of the streamlit UI are provided below:

Upload Data

Upload combined CSV

Drag and drop file here
Limit 200MB per file • CSV

Browse files

cleaned_combined.csv
1.2MB

Upload insights JSON

Drag and drop file here
Limit 200MB per file • JSON

Browse files

insights.json
3.9KB

Loaded CSV with 10869 rows.

Loaded insights.json

Deploy

App Market Intelligence Dashboard

Data Preview

	app	category	rating	reviews	type	price	genres	last_upd
0	Photo Editor & Candy Camera & Grid & ScrapBook	['ART', 'DESIGN']	4.1	159	Free	0	['Art & Design']	2018-01
1	Coloring book moana	['ART', 'DESIGN']	3.9	967	Free	0	['Art & Design', 'Pretend Play']	2018-01
2	U Launcher Lite – FREE Live Cool Themes, Hide Apps	['ART', 'DESIGN']	4.7	87510	Free	0	['Art & Design']	2018-06
3	Sketch - Draw & Paint	['ART', 'DESIGN']	4.5	215644	Free	0	['Art & Design']	2018-06
4	Pixel Draw - Number Art Coloring Book	['ART', 'DESIGN']	4.3	967	Free	0	['Art & Design', 'Creativity']	2018-06
5	Paper flowers instructions	['ART', 'DESIGN']	4.4	167	Free	0	['Art & Design']	2017-03
6	Smoke Effect Photo Maker - Smoke Editor	['ART', 'DESIGN']	3.8	178	Free	0	['Art & Design']	2018-04
7	Infinite Painter	['ART', 'DESIGN']	4.1	36815	Free	0	['Art & Design']	2018-06
8	Garden Coloring Book	['ART', 'DESIGN']	4.4	13791	Free	0	['Art & Design']	2017-09
9	Kids Paint Free - Drawing Fun	['ART', 'DESIGN']	4.7	121	Free	0	['Art & Design', 'Creativity']	2018-07

Insights (from insights.json)

Best performing category (by reviews)

Free vs Paid apps

Upload Data

Upload combined CSV

Drag and drop file here
Limit 200MB per file • CSV

Browse files

cleaned_combined.csv
1.2MB

Upload insights JSON

Drag and drop file here
Limit 200MB per file • JSON

Browse files

insights.json
3.9KB

Loaded CSV with 10869 rows.

Loaded insights.json

Deploy

Insights (from insights.json)

Best performing category (by reviews)

Free vs Paid apps

Top 5 categories by app count

Most expensive app category

Content rating distribution

Highest rated category

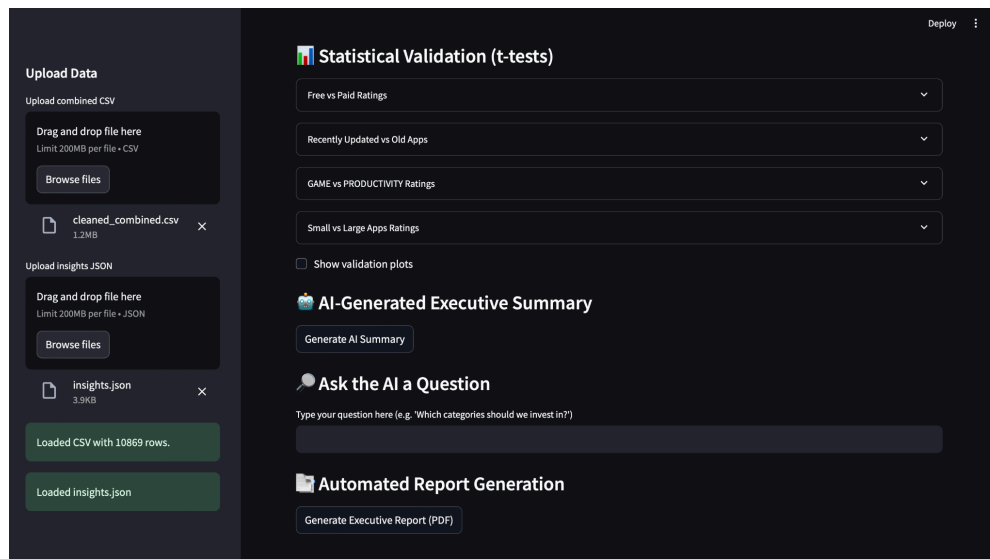
Lowest rated category

Most popular genre

Store comparison

Recently updated vs older apps

Statistical Validation (t-tests)



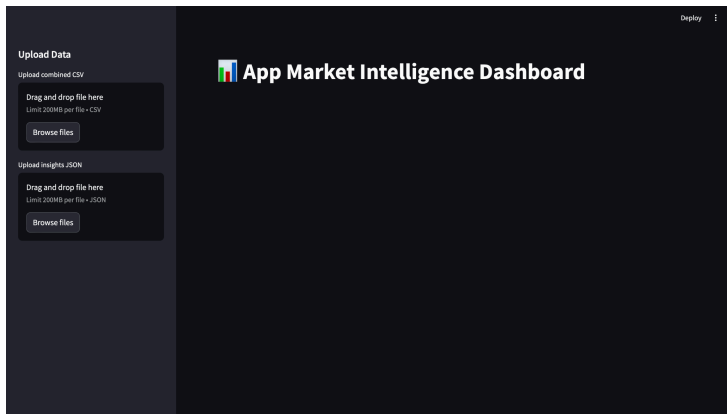
The Organisation of the files provided:

AI-Powered Market Intelligence (Base Directory):

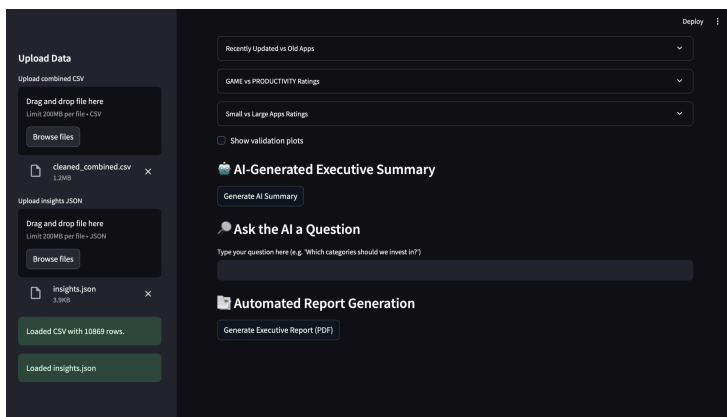
- > Cleaning playstore dataset.ipynb
- > Updated Combine Datasets.ipynb
- > latest_appstore_integ.py
- > updated_streamlit_app.py
- > json_creation.py
- > updated_insights.json
- > dataset
 - > dataset/appstore_apps_by_name_1.csv
 - > dataset/appstore_apps_by_name_2.csv
 - > dataset/appstore_apps_by_name_3.csv
 - > dataset/appstore_apps_by_name_4.csv
 - > dataset/updated_cleaned_combined.csv
 - > dataset/cleaned_googleplaystore.csv
 - > dataset/archive-2
 - > dataset/archive-2/googleplaystore.csv

STEPS TO RUN THE STREAMLIT APP:

1. The first step is to organise all the files provided in the exact hierarchy provided.
2. After that, open the terminal at the root directory.
3. Then run the command "streamlit run updated_streamlit_app.py"
4. It opens the UI where it asks to upload the combined csv file (**Updated Combine Datasets.ipynb**) and the insights json file (**updated_insights.json**) first. Like given in the image below.



5. Then after uploading you can use all the functionality of app and to create an automated report of the data, press on the Generate Executive Report button.



6. After that when the report is ready you will see a Download Report button. Clicking that downloads the report to the device.

